JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.stream

Interface LongStream

All Superinterfaces:

AutoCloseable, BaseStream<Long,LongStream>

```
public interface LongStream
extends BaseStream<Long,LongStream>
```

A sequence of primitive long-valued elements supporting sequential and parallel aggregate operations. This is the `long` primitive specialization of `Stream`.

The following example illustrates an aggregate operation using `Stream` and `LongStream`, computing the sum of the weights of the red widgets:

```
long sum = widgets.stream()
                  .filter(w -> w.getColor() == RED)
                  .mapToLong(w -> w.getWeight())
                  .sum();
```

See the class documentation for `Stream` and the package documentation for java.util.stream for additional specification of streams, stream operations, stream pipelines, and parallelism.

Since:

1.8

See Also:

Stream, java.util.stream

Nested Class Summary

Nested Classes	Interface	Description
static interface	LongStream.Builder	A mutable builder for a LongStream.

Method Summary

Default M

Modifier and Type	Method	Description
boolean	allMatch(LongPredicate predicate)	Returns wheth elements of th match the prov predicate.
boolean	anyMatch(LongPredicate predicate)	Returns wheth elements of th match the prov predicate.
DoubleStream	asDoubleStream()	Returns a Doub consisting of t elements of th converted to d
OptionalDouble	average()	Returns an OptionalDoub describing the arithmetic mea elements of th or an empty op this stream is
Stream<Long>	boxed()	Returns a Stre consisting of t elements of th each boxed to
static LongStream.Builder	builder()	Returns a build LongStream.
<R> R	collect(Supplier<R> supplier, ObjLongConsumer<R> accumulator, BiConsumer<R,R> combiner)	Performs a mu reduction ope the elements o stream.
static LongStream	concat(LongStream a, LongStream b)	

		Creates a lazily concatenated s whose element the elements o stream followe the elements o second stream
long	**count**()	Returns the co elements in thi
LongStream	**distinct**()	Returns a strea consisting of th elements of thi
static **LongStream**	**empty**()	Returns an em sequential Lon
LongStream	**filter**(**LongPredicate** predicate)	Returns a strea consisting of th elements of thi that match the predicate.
OptionalLong	**findAny**()	Returns an **OptionalLong** describing som element of the or an empty OptionalLong stream is empt
OptionalLong	**findFirst**()	Returns an **OptionalLong** describing the element of this or an empty OptionalLong stream is empt
LongStream	**flatMap**(**LongFunction**<? extends **LongStream**> mapper)	Returns a strea consisting of th of replacing ea element of this with the conte mapped strean produced by ap the provided m

		function to eac element.
void	**forEach**(**LongConsumer** action)	Performs an ac each element o stream.
void	**forEachOrdered**(**LongConsumer** action)	Performs an ac each element o stream, guaran that each elem processed in e order for strea have a defined encounter ord
static **LongStream**	**generate**(**LongSupplier** s)	Returns an infi sequential unor stream where element is gen the provided `LongSupplier`
static **LongStream**	**iterate**(long seed, **LongUnaryOperator** f)	Returns an infi sequential ord `LongStream` pr by iterative ap of a function `f` initial element producing a `St` consisting of `s` `f(seed)`, `f(f(` etc.
PrimitiveIterator.OfLong	**iterator**()	Returns an iter the elements o stream.
LongStream	**limit**(long maxSize)	Returns a strea consisting of th elements of thi truncated to be longer than `ma` length.

LongStream	map(LongUnaryOperator mapper)	Returns a stre consisting of t of applying the function to the of this stream.
DoubleStream	mapToDouble(LongToDoubleFunction mapper)	Returns a Doub consisting of t of applying the function to the of this stream.
IntStream	mapToInt(LongToIntFunction mapper)	Returns an Int consisting of t of applying the function to the of this stream.
<U> Stream<U>	mapToObj(LongFunction<? extends U> mapper)	Returns an obj valued Stream consisting of t of applying the function to the of this stream.
OptionalLong	max()	Returns an OptionalLong describing the element of this or an empty op this stream is
OptionalLong	min()	Returns an OptionalLong describing the element of this or an empty op this stream is
boolean	noneMatch(LongPredicate predicate)	Returns wheth elements of th match the prov predicate.
static LongStream	of(long... values)	Returns a sequ ordered strean elements are t specified value

static **LongStream**	**of**(long t)	Returns a sequ LongStream co single element
LongStream	**parallel**()	Returns an equ stream that is
LongStream	**peek**(**LongConsumer** action)	Returns a strea consisting of th elements of thi additionally pe the provided a each element a elements are c from the result stream.
static **LongStream**	**range**(long startInclusive, long endExclusive)	Returns a sequ ordered LongS from startInc (inclusive) to endExclusive (exclusive) by a incremental st
static **LongStream**	**rangeClosed**(long startInclusive, long endInclusive)	Returns a sequ ordered LongS from startInc (inclusive) to endInclusive (inclusive) by a incremental st
OptionalLong	**reduce**(**LongBinaryOperator** op)	Performs a **red** on the element stream, using a **associative** accumulation f and returns an OptionalLong describing the value, if any.
long	**reduce**(long identity, **LongBinaryOperator** op)	Performs a **red** on the element stream, using t provided ident and an **associa** accumulation f

		and returns th value.
LongStream	sequential()	Returns an equ stream that is sequential.
LongStream	skip(long n)	Returns a strea consisting of th remaining elem this stream aft discarding the elements of th
LongStream	sorted()	Returns a strea consisting of th elements of thi in sorted order
Spliterator.OfLong	spliterator()	Returns a split the elements o stream.
long	sum()	Returns the su elements in thi
LongSummaryStatistics	summaryStatistics()	Returns a LongSummaryS describing var summary data elements of thi
long[]	toArray()	Returns an arr containing the of this stream.

Methods inherited from interface java.util.stream.BaseStream

close, isParallel, onClose, unordered

Method Detail

filter

```
LongStream filter(LongPredicate predicate)
```

Returns a stream consisting of the elements of this stream that match the given predicate.

This is an intermediate operation.

Parameters:

`predicate - a non-interfering, stateless predicate to apply to each element to determine if it should be included`

Returns:

`the new stream`

map

`LongStream map(LongUnaryOperator mapper)`

Returns a stream consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToObj

`<U> Stream<U> mapToObj(LongFunction<? extends U> mapper)`

Returns an object-valued `Stream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Type Parameters:

`U - the element type of the new stream`

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToInt

`IntStream mapToInt(LongToIntFunction mapper)`

Returns an `IntStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

mapToDouble

`DoubleStream mapToDouble(LongToDoubleFunction mapper)`

Returns a `DoubleStream` consisting of the results of applying the given function to the elements of this stream.

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element`

Returns:

`the new stream`

flatMap

`LongStream flatMap(LongFunction<? extends LongStream> mapper)`

Returns a stream consisting of the results of replacing each element of this stream with the contents of a mapped stream produced by applying the provided mapping function to each element. Each mapped stream is `closed` after its contents have been placed into this stream. (If a mapped stream is `null` an empty stream is used, instead.)

This is an intermediate operation.

Parameters:

`mapper - a non-interfering, stateless function to apply to each element which produces a LongStream of new values`

Returns:

the new stream

See Also:

Stream.flatMap(Function)

distinct

LongStream distinct()

Returns a stream consisting of the distinct elements of this stream.

This is a stateful intermediate operation.

Returns:

the new stream

sorted

LongStream sorted()

Returns a stream consisting of the elements of this stream in sorted order.

This is a stateful intermediate operation.

Returns:

the new stream

peek

LongStream peek(LongConsumer action)

Returns a stream consisting of the elements of this stream, additionally performing the provided action on each element as elements are consumed from the resulting stream.

This is an intermediate operation.

For parallel stream pipelines, the action may be called at whatever time and in whatever thread the element is made available by the upstream operation. If the action modifies shared state, it is responsible for providing the required synchronization.

API Note:

This method exists mainly to support debugging, where you want to see the elements as they flow past a certain point in a pipeline:

```
    LongStream.of(1, 2, 3, 4)
```

```
        .filter(e -> e > 2)
        .peek(e -> System.out.println("Filtered value: " + e))
        .map(e -> e * e)
        .peek(e -> System.out.println("Mapped value: " + e))
        .sum();
```

Parameters:

`action - a non-interfering action to perform on the elements as they are consumed from the stream`

Returns:

`the new stream`

limit

`LongStream limit(long maxSize)`

Returns a stream consisting of the elements of this stream, truncated to be no longer than `maxSize` in length.

This is a short-circuiting stateful intermediate operation.

API Note:

`While limit() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of maxSize, since limit(n) is constrained to return not just any` *`n`* `elements, but the` *`first n`* `elements in the encounter order. Using an unordered stream source (such as generate(LongSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of limit() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with limit() in parallel pipelines, switching to sequential execution with sequential() may improve performance.`

Parameters:

`maxSize - the number of elements the stream should be limited to`

Returns:

`the new stream`

Throws:

`IllegalArgumentException - if maxSize is negative`

skip

`LongStream skip(long n)`

Returns a stream consisting of the remaining elements of this stream after discarding the first `n` elements of the stream. If this stream contains fewer than `n` elements then an empty stream will be returned.

This is a stateful intermediate operation.

API Note:

While skip() is generally a cheap operation on sequential stream pipelines, it can be quite expensive on ordered parallel pipelines, especially for large values of n, since skip(n) is constrained to skip not just any *n* elements, but the *first n* elements in the encounter order. Using an unordered stream source (such as generate(LongSupplier)) or removing the ordering constraint with BaseStream.unordered() may result in significant speedups of skip() in parallel pipelines, if the semantics of your situation permit. If consistency with encounter order is required, and you are experiencing poor performance or memory utilization with skip() in parallel pipelines, switching to sequential execution with sequential() may improve performance.

Parameters:

n - the number of leading elements to skip

Returns:

the new stream

Throws:

IllegalArgumentException - if n is negative

forEach

`void forEach(LongConsumer action)`

Performs an action for each element of this stream.

This is a terminal operation.

For parallel stream pipelines, this operation does *not* guarantee to respect the encounter order of the stream, as doing so would sacrifice the benefit of parallelism. For any given element, the action may be performed at whatever time and in whatever thread the library chooses. If the action accesses shared state, it is responsible for providing the required synchronization.

Parameters:

action - a non-interfering action to perform on the elements

forEachOrdered

```
void forEachOrdered(LongConsumer action)
```

Performs an action for each element of this stream, guaranteeing that each element is processed in encounter order for streams that have a defined encounter order.

This is a terminal operation.

Parameters:

`action` - a non-interfering action to perform on the elements

See Also:

`forEach(LongConsumer)`

toArray

```
long[] toArray()
```

Returns an array containing the elements of this stream.

This is a terminal operation.

Returns:

an array containing the elements of this stream

reduce

```
long reduce(long identity,
            LongBinaryOperator op)
```

Performs a reduction on the elements of this stream, using the provided identity value and an associative accumulation function, and returns the reduced value. This is equivalent to:

```
long result = identity;
for (long element : this stream)
    result = accumulator.applyAsLong(result, element)
return result;
```

but is not constrained to execute sequentially.

The `identity` value must be an identity for the accumulator function. This means that for all `x`, `accumulator.apply(identity, x)` is equal to `x`. The `accumulator` function must be an associative function.

This is a terminal operation.

API Note:

Sum, min, max, and average are all special cases of reduction. Summing a stream of numbers can be expressed as:

```
long sum = integers.reduce(0, (a, b) -> a+b);
```

or more compactly:

```
long sum = integers.reduce(0, Long::sum);
```

While this may seem a more roundabout way to perform an aggregation compared to simply mutating a running total in a loop, reduction operations parallelize more gracefully, without needing additional synchronization and with greatly reduced risk of data races.

Parameters:

identity - the identity value for the accumulating function

op - an associative, non-interfering, stateless function for combining two values

Returns:

the result of the reduction

See Also:

sum(), min(), max(), average()

reduce

`OptionalLong reduce(LongBinaryOperator op)`

Performs a reduction on the elements of this stream, using an associative accumulation function, and returns an `OptionalLong` describing the reduced value, if any. This is equivalent to:

```
boolean foundAny = false;
long result = null;
for (long element : this stream) {
    if (!foundAny) {
        foundAny = true;
        result = element;
    }
    else
```

```
            result = accumulator.applyAsLong(result, element);
    }
    return foundAny ? OptionalLong.of(result) : OptionalLong.empty();
```

but is not constrained to execute sequentially.

The `accumulator` function must be an associative function.

This is a terminal operation.

Parameters:

`op - an associative, non-interfering, stateless function for combining two values`

Returns:

`the result of the reduction`

See Also:

`reduce(long, LongBinaryOperator)`

collect

```
<R> R collect(Supplier<R> supplier,
              ObjLongConsumer<R> accumulator,
              BiConsumer<R,R> combiner)
```

Performs a mutable reduction operation on the elements of this stream. A mutable reduction is one in which the reduced value is a mutable result container, such as an `ArrayList`, and elements are incorporated by updating the state of the result rather than by replacing the result. This produces a result equivalent to:

```
    R result = supplier.get();
    for (long element : this stream)
        accumulator.accept(result, element);
    return result;
```

Like `reduce(long, LongBinaryOperator)`, `collect` operations can be parallelized without requiring additional synchronization.

This is a terminal operation.

Type Parameters:

`R - type of the result`

Parameters:

supplier - a function that creates a new result container. For a parallel execution, this function may be called multiple times and must return a fresh value each time.

accumulator - an associative, non-interfering, stateless function for incorporating an additional element into a result

combiner - an associative, non-interfering, stateless function for combining two values, which must be compatible with the accumulator function

Returns:

the result of the reduction

See Also:

Stream.collect(Supplier, BiConsumer, BiConsumer)

sum

long sum()

Returns the sum of elements in this stream. This is a special case of a reduction and is equivalent to:

```
return reduce(0, Long::sum);
```

This is a terminal operation.

Returns:

the sum of elements in this stream

min

`OptionalLong min()`

Returns an `OptionalLong` describing the minimum element of this stream, or an empty optional if this stream is empty. This is a special case of a reduction and is equivalent to:

```
return reduce(Long::min);
```

This is a terminal operation.

Returns:

an `OptionalLong` containing the minimum element of this stream, or an empty `OptionalLong` if the stream is empty

max

`OptionalLong max()`

Returns an `OptionalLong` describing the maximum element of this stream, or an empty optional if this stream is empty. This is a special case of a reduction and is equivalent to:

```
return reduce(Long::max);
```

This is a terminal operation.

Returns:

an `OptionalLong` containing the maximum element of this stream, or an empty `OptionalLong` if the stream is empty

count

`long count()`

Returns the count of elements in this stream. This is a special case of a reduction and is equivalent to:

```
return map(e -> 1L).sum();
```

This is a terminal operation.

Returns:

the count of elements in this stream

average

`OptionalDouble average()`

Returns an `OptionalDouble` describing the arithmetic mean of elements of this stream, or an empty optional if this stream is empty. This is a special case of a reduction.

This is a terminal operation.

Returns:

an `OptionalDouble` containing the average element of this stream, or an empty optional if the stream is empty

summaryStatistics

`LongSummaryStatistics summaryStatistics()`

Returns a `LongSummaryStatistics` describing various summary data about the elements of this stream. This is a special case of a reduction.

This is a terminal operation.

Returns:

a `LongSummaryStatistics` describing various summary data about the elements of this stream

anyMatch

`boolean anyMatch(LongPredicate predicate)`

Returns whether any elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then `false` is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *existential quantification* of the predicate over the elements of the stream (for some x P(x)).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

true if any elements of the stream match the provided predicate, otherwise false

allMatch

boolean allMatch(LongPredicate predicate)

Returns whether all elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the predicate over the elements of the stream (for all x P(x)). If the stream is empty, the quantification is said to be *vacuously satisfied* and is always true (regardless of P(x)).

Parameters:

predicate - a non-interfering, stateless predicate to apply to elements of this stream

Returns:

true if either all elements of the stream match the provided predicate or the stream is empty, otherwise false

noneMatch

boolean noneMatch(LongPredicate predicate)

Returns whether no elements of this stream match the provided predicate. May not evaluate the predicate on all elements if not necessary for determining the result. If the stream is empty then true is returned and the predicate is not evaluated.

This is a short-circuiting terminal operation.

API Note:

This method evaluates the *universal quantification* of the negated predicate over the elements of the stream (for all x ~P(x)). If the stream is empty, the quantification is said to be vacuously satisfied and is always true, regardless of P(x).

Parameters:

`predicate - a non-interfering, stateless predicate to apply to elements of this stream`

Returns:

`true if either no elements of the stream match the provided predicate or the stream is empty, otherwise false`

findFirst

`OptionalLong findFirst()`

Returns an `OptionalLong` describing the first element of this stream, or an empty `OptionalLong` if the stream is empty. If the stream has no encounter order, then any element may be returned.

This is a short-circuiting terminal operation.

Returns:

`an OptionalLong describing the first element of this stream, or an empty OptionalLong if the stream is empty`

findAny

`OptionalLong findAny()`

Returns an `OptionalLong` describing some element of the stream, or an empty `OptionalLong` if the stream is empty.

This is a short-circuiting terminal operation.

The behavior of this operation is explicitly nondeterministic; it is free to select any element in the stream. This is to allow for maximal performance in parallel operations; the cost is that multiple invocations on the same source may not return the same result. (If a stable result is desired, use `findFirst()` instead.)

Returns:

`an OptionalLong describing some element of this stream, or an empty OptionalLong if the stream is empty`

See Also:

`findFirst()`

asDoubleStream

`DoubleStream asDoubleStream()`

Returns a `DoubleStream` consisting of the elements of this stream, converted to `double`.

This is an intermediate operation.

Returns:

`a DoubleStream consisting of the elements of this stream, converted to double`

boxed

`Stream<Long> boxed()`

Returns a `Stream` consisting of the elements of this stream, each boxed to a `Long`.

This is an intermediate operation.

Returns:

`a Stream consistent of the elements of this stream, each boxed to Long`

sequential

`LongStream sequential()`

Description copied from interface: `BaseStream`

Returns an equivalent stream that is sequential. May return itself, either because the stream was already sequential, or because the underlying stream state was modified to be sequential.

This is an intermediate operation.

Specified by:

`sequential in interface BaseStream<Long,LongStream>`

Returns:

`a sequential stream`

parallel

`LongStream parallel()`

Description copied from interface: BaseStream
Returns an equivalent stream that is parallel. May return itself, either because the stream was already parallel, or because the underlying stream state was modified to be parallel.

This is an intermediate operation.

Specified by:

`parallel in interface BaseStream<Long,LongStream>`

Returns:

a parallel stream

iterator

`PrimitiveIterator.OfLong iterator()`

Description copied from interface: BaseStream
Returns an iterator for the elements of this stream.

This is a terminal operation.

Specified by:

`iterator in interface BaseStream<Long,LongStream>`

Returns:

the element iterator for this stream

spliterator

`Spliterator.OfLong spliterator()`

Description copied from interface: BaseStream
Returns a spliterator for the elements of this stream.

This is a terminal operation.

Specified by:

`spliterator in interface BaseStream<Long,LongStream>`

Returns:

the element spliterator for this stream

builder

```
static LongStream.Builder builder()
```

Returns a builder for a `LongStream`.

Returns:

a stream builder

empty

```
static LongStream empty()
```

Returns an empty sequential `LongStream`.

Returns:

an empty sequential stream

of

```
static LongStream of(long t)
```

Returns a sequential `LongStream` containing a single element.

Parameters:

t - the single element

Returns:

a singleton sequential stream

of

```
static LongStream of(long... values)
```

Returns a sequential ordered stream whose elements are the specified values.

Parameters:

values - the elements of the new stream

Returns:

the new stream

iterate

```
static LongStream iterate(long seed,
                          LongUnaryOperator f)
```

Returns an infinite sequential ordered `LongStream` produced by iterative application of a function `f` to an initial element `seed`, producing a `Stream` consisting of `seed`, `f(seed)`, `f(f(seed))`, etc.

The first element (position `0`) in the `LongStream` will be the provided `seed`. For `n > 0`, the element at position `n`, will be the result of applying the function `f` to the element at position `n - 1`.

Parameters:

`seed - the initial element`

`f - a function to be applied to the previous element to produce a new element`

Returns:

`a new sequential LongStream`

generate

```
static LongStream generate(LongSupplier s)
```

Returns an infinite sequential unordered stream where each element is generated by the provided `LongSupplier`. This is suitable for generating constant streams, streams of random elements, etc.

Parameters:

`s - the LongSupplier for generated elements`

Returns:

`a new infinite sequential unordered LongStream`

range

```
static LongStream range(long startInclusive,
                        long endExclusive)
```

Returns a sequential ordered `LongStream` from `startInclusive` (inclusive) to `endExclusive` (exclusive) by an incremental step of `1`.

API Note:

`An equivalent sequence of increasing values can be produced sequentially using a for loop as follows:`

```
for (long i = startInclusive; i < endExclusive ; i++) { ... }
```

Parameters:

`startInclusive` - the (inclusive) initial value

`endExclusive` - the exclusive upper bound

Returns:

a sequential LongStream for the range of long elements

rangeClosed

```
static LongStream rangeClosed(long startInclusive,
                              long endInclusive)
```

Returns a sequential ordered `LongStream` from `startInclusive` (inclusive) to `endInclusive` (inclusive) by an incremental step of 1.

API Note:

An equivalent sequence of increasing values can be produced sequentially using a for loop as follows:

```
for (long i = startInclusive; i <= endInclusive ; i++) { ... }
```

Parameters:

`startInclusive` - the (inclusive) initial value

`endInclusive` - the inclusive upper bound

Returns:

a sequential LongStream for the range of long elements

concat

```
static LongStream concat(LongStream a,
                         LongStream b)
```

Creates a lazily concatenated stream whose elements are all the elements of the first stream followed by all the elements of the second stream. The resulting stream is ordered if both of the input streams are ordered, and parallel if either of the input streams is parallel. When the resulting stream is closed, the close handlers for both input streams are invoked.

Implementation Note:

Use caution when constructing streams from repeated concatenation. Accessing an element of a deeply concatenated stream can result in deep call chains, or even StackOverflowException.

Parameters:

a - the first stream

b - the second stream

Returns:

the concatenation of the two input streams

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.